

AURA beyond the Mac.

This document explains how AURA moves from a Mac-hosted prototype into a real user app: public backend, persistent memory, voice infrastructure, TestFlight, App Store, privacy, costs, and operations.

The prototype proved the feeling: AURA can listen, remember, recall, remind, and speak. The launch plan turns that feeling into infrastructure people can trust.

Now

Mac backend for fast iteration

Alpha

Public HTTPS backend + Supabase memory

Beta

Always-on VM or VPS for Pocket TTS

Launch

TestFlight, App Store, billing, privacy



What exists today.

AURA currently works as a personal prototype: iPhone app installed locally, backend running on the Mac, and the phone calls the LAN URL. That is perfect for building fast, but not shippable to users.

Mobile app

Expo/React Native iOS app with tap-to-speak, Type Instead, reminders, local notification scheduling, minimal AURA UI, and current bundle id **ai.aura.journal.recall**.

Mac backend

Express API on **0.0.0.0:4000**. It handles Groq STT, OpenRouter replies, Gemini/Pocket/iOS voice chain, memory, reminders, and emotional continuity.

Memory

Raw archive, smart memories, session summaries, emotional signals, theme scores, relationship continuity, exact date recall, and local-file fallback are already built.

Voice

Gemini TTS is premium. Pocket TTS now works as quota-safe fallback. iOS speech is emergency fallback only.

Reminders

Foreground reminders speak and show text. Background/closed app gets normal AURA text notifications.

Limitation

Everything depends on the Mac being online and on the same network. Users cannot rely on **192.168.1.7**.

Decision: keep the Mac backend for now because it is fast, controllable, and cheap while the product feel is still being shaped.

The real app shape.

AURA needs a public HTTPS backend URL, a durable database, secure secrets, and enough compute to run the orchestration layer without the Mac.

iPhone app

01

The IPA/App Store app should call `https://api.aura...`, not a LAN IP. Later, remote config should let the app learn a backend URL without a rebuild.

Backend API

02

The Node/Express API remains the orchestration brain: STT, LLM, TTS, memory, reminders, provider health, rate limits, and user/account logic.

Database

03

Supabase Postgres stores raw conversation archive, smart memories, reminders, check-ins, emotional signals, theme scores, and relationship continuity.

Provider layer

04

Groq, OpenRouter, Gemini, Pocket TTS, and future providers remain behind the backend. Secrets never ship inside the IPA.

Observability

05

Every critical path needs logs and health metrics: STT, LLM, voice provider used, reminder delivery, memory writes, and fallback frequency.

Production target request path:

```
AURA iOS app
-> HTTPS API gateway / Express backend
-> Groq Whisper for transcription
-> OpenRouter for reasoning
-> Gemini TTS, Pocket TTS, or iOS fallback
-> Supabase Postgres for memory + reminders
-> app receives reply text + optional voice audio
```

What to use when.

The hardest requirement is not “host a Node API.” The hard part is “host a Node API that can also do audio, memory, and Pocket TTS reliably without charging too early.”

Mac now	Best for daily iteration. Zero deployment friction. Bad for users because the Mac and Wi-Fi must stay online.
Render Free	Easy deploy and existing <code>render.yaml</code> . Free web services sleep after idle and filesystem is ephemeral, so Supabase is required. Pocket TTS may be slow/unreliable.
Koyeb Free	One free service with small CPU/RAM. Better for simple API tests than heavy voice fallback. Good alpha experiment.
Railway	Developer-friendly, but truly useful always-on usually means usage credits or Hobby plan. Good once small spend is acceptable.
Vercel	Great for web apps and functions. Awkward for always-running Express + file uploads + Pocket TTS + long audio generation. Not recommended as main AURA backend.
Oracle/VPS	Best serious-beta choice. A real VM can stay on, run PM2/systemd, install Pocket TTS, keep logs, and be updated with git pull.

Recommendation: Mac now. Render/Koyeb only for quick public URL tests. Oracle Always Free VM or low-cost VPS for serious beta. Paid managed hosting once real users justify it.

A practical rollout ladder.

Do not jump straight from a local prototype to a public App Store launch. AURA needs reliability gates because memory and voice are the magic.

Now**Local Mac prototype**

Keep improving product feel. Use Mac backend, existing IPA builds, local Pocket TTS, and local-file fallback while testing daily.

A1**Private alpha URL**

Deploy backend to Render/Koyeb/temporary host with Supabase. Rebuild IPA to point to HTTPS. Accept sleep/cold-start while validating remote behavior.

A2**Always-on beta backend**

Move API to Oracle Always Free VM or a cheap VPS. Install Node, uv/uvx, Pocket TTS, PM2/systemd, Caddy/Nginx HTTPS, and log rotation.

B1**TestFlight beta**

Create Apple developer account, bundle id, signing, privacy policy, app metadata, crash logging, and invite 10-50 trusted users.

B2**Paid/limited launch**

Add accounts, usage caps, export/delete memory, support channel, analytics, and billing/BYOK before wider release.

P**Production**

Move to paid reliable infrastructure, separate staging/prod, backups, monitoring, alerts, provider failover, and incident runbooks.

What the server must become.

The current Express backend can evolve into production without a rewrite. It needs durability, account boundaries, and operational guardrails.

Public HTTPS

Expose the API through a domain like `api.aura....`. Use Caddy/Nginx or platform HTTPS. No IPA should point to local IP.

Environment secrets

OpenRouter, Groq, Gemini, Supabase, and future provider keys live only in backend env vars or secret manager.

Memory isolation

Every archive turn, smart memory, reminder, signal, and relationship record must be tied to a real user id/account.

Rate limits

Per-user daily limits protect provider credits. Beta users can have invite codes and known limits.

Provider health

`/health` should show active key counts, cooldowns, fallback readiness, and memory reachability without leaking secrets.

Deployment flow

GitHub push -> build -> deploy. For VM: SSH/git pull/npm build/PM2 reload. For platforms: auto-deploy with environment variables.

Minimum production backend checklist:

- HTTPS domain
- Supabase production project
- user identity / anonymous id migration plan
- per-user memory rows
- provider key rotation
- logs + health + uptime checks
- no local filesystem dependency for long-term data
- backup/export/delete memory endpoints

The memory must be trustworthy.

AURA's core magic is long-term recall. Production memory cannot be a local JSON file. It needs clear layers, privacy controls, and user trust.

Raw archive	Source of truth. Every user/AURA turn with timestamp, timezone, role, session id, searchable text, and metadata.
Smart memories	Summaries, topics, goals, emotions, importance, embeddings, and memory layer. Used for normal context.
Session summaries	Compact summaries after idle gaps so old conversations can be recalled without giant prompts.
Emotional continuity	Derived signals and trends. Never overwrite archive truth. Use restraint: subtle companion context, not robotic timestamp dumps.
User controls	Before public launch: export my data, delete my data, pause memory, clear a memory, and privacy policy explaining storage.

Launch rule: exact date-wise recall must stay deterministic; emotional intelligence is derived context on top, not replacement truth.

Premium voice without bankrupting the app.

Voice is expensive and fragile if provider limits are ignored. AURA needs a clear hierarchy and telemetry before users arrive.

1

Gemini TTS premium path

Use Gemini for the best AURA voice when keys/quota are healthy. Track cooldowns and failures.

2

Pocket TTS fallback

Keep Pocket TTS as a no-provider-quota backend fallback. Best on a VM/VPS where model/runtime can stay warm.

3

iOS speech emergency

Use device speech only if backend voice fails. It should be loud and normal, but not the main brand voice.

4

Language routing

For Hindi/Hinglish, Gemini is best. Pocket English can sound accented. Later route Hindi-heavy fallback directly to a better Hindi provider or iOS **hi-IN**.

5

Voice metrics

Store provider used, latency, error type, and fallback count. This tells when paid TTS is necessary.

Voice strategy at launch:

Normal chat: Gemini -> Pocket -> iOS

Foreground reminder: Gemini -> Pocket -> iOS

Background reminder: notification banner only

Do not promise background spoken voice. iOS will not reliably let the app speak while killed/closed.

Useful, honest, reliable.

AURA reminders should feel personal, but the product must be honest about iOS limits. Foreground voice is magical; background delivery is notification-only.

Foreground

If app is open at reminder time, AURA shows the reminder text and speaks it through Gemini/Pocket/iOS. This is the premium experience.

Background/closed

iOS local notification banner appears from the AURA app. No custom voice promise. No fragile custom notification sound requirement.

Check-ins

Self-improvement prompts like “ask me in 30 minutes if I’m still smoking” become check-ins, and answers are saved as memory context.

Production reliability

Reminders need server record + local notification id + status. Future multi-device support needs cloud sync and conflict handling.

UX wording: when setting a foreground-style reminder, AURA can say: “I’ll remind you. Keep AURA open around then if you want me to speak it.”

From IPA to real users.

Sideloadly is for prototype testing. Real users require TestFlight/App Store, stable signing, privacy disclosures, and support.

01

Apple developer account

Enroll, create bundle id, app record, signing certificates/profiles, and TestFlight pipeline.

02

Production backend URL

Build app with public HTTPS API URL. Add remote config later so backend URL changes do not require a full rebuild.

03

Privacy policy

Explain voice transcription, conversation archive, memory storage, reminders, provider processing, data deletion, and support contact.

04

Crash/error reporting

Add Sentry/Expo style crash logs and backend request ids so failed voice/memory flows can be debugged.

05

TestFlight beta

Invite trusted users first. Watch provider cost, memory correctness, reminder reliability, and emotional UX.

06

App Store release

Submit only after backend uptime, privacy controls, support email, and usage limits are ready.

AURA cannot pay for everyone silently.

The launch model must protect provider costs. Voice + LLM + STT can become expensive if every user has unlimited usage.

Cost drivers	OpenRouter LLM, Groq STT, Gemini TTS, backend compute, database storage, logs, egress, App Store developer program.
Free beta	Invite-only, usage-limited, Rehan-funded. Keep daily message/voice caps and monitor fallback usage.
BYOK	Power users add their own OpenRouter/Groq/Gemini keys. Reduces cost burden, but UX must validate and store securely.
Subscription	Normal users pay for hosted usage. Best long-term UX, but requires billing, quotas, and clear plan limits.
Hybrid	Recommended: free trial with limits, BYOK option, and paid hosted plan when public launch starts.

Simple rule: until revenue exists, do not offer unlimited hosted AI voice/memory to public users.

Memory apps need unusual care.

AURA stores intimate life context. Trust is not decoration; it is core product infrastructure.

Clear consent

Tell users AURA saves conversations and summaries so it can remember. No buried surprise.

Memory controls

Pause memory, delete one memory, delete all data, export archive, and disable emotional continuity.

Data boundaries

Never leak memories across users. Every memory query must filter by user id before ranking.

Sensitive topics

Avoid diagnosis, manipulation, or overconfident psychological labels. Use gentle language.

Retention

Define how long raw archives and summaries live. Let users clear old archive if they want.

Provider disclosure

Explain that audio/text may be processed by AI providers like Groq, OpenRouter, Gemini, or future services.

How AURA stays alive.

AURA needs a small operating system around it: health checks, logs, backups, alerts, and deploy discipline.

Health: public `/health` shows provider status, memory database reachability, archive readiness, and voice fallback readiness.

Logs: request id, user id hash, route, latency, provider used, error class, voice provider, reminder id, memory write result.

Alerts: backend down, OpenRouter all keys cooling, Gemini all keys cooling, Pocket TTS failing, Supabase unreachable, reminders not delivering.

Backups: Supabase backups/export, periodic archive dump, migration scripts, and memory restore test.

Deploys: staging before prod, rollback path, env var checklist, and no deploy right before important testing.

Support: user-visible error messages should be calm and short, not “brain broken” unless debugging.

What changes from local to cloud.

The codebase does not need a full rewrite. The migration is mostly configuration, persistence, auth, and deployment hardening.

API URL	Replace <code>http://192.168.1.7:4000</code> with public HTTPS in mobile env. Later add remote config and multiple API candidates.
Storage	Move from local JSON fallback to Supabase tables for archive, memories, reminders, theme scores, relationships, todos, and usage.
Auth	Start with anonymous durable user id, then add Apple Sign In or email login for multi-device sync and account recovery.
Pocket TTS	Prefer VM/VPS. Install <code>uvx</code> , warm model cache, set timeout, monitor CPU, and make fallback non-blocking.
Rebuild	IPA/App Store build required when API URL or native capabilities change. Backend-only changes deploy without app rebuild.

```
Minimum cloud env vars:
PORT=4000
HOST=0.0.0.0
APP_ORIGIN=*
SUPABASE_URL=...
SUPABASE_SERVICE_ROLE_KEY=...
OPENROUTER_API_KEYS=...
OPENROUTER_MODEL=z-ai/glm-4.5-air:free
GROQ_API_KEYS=...
GEMINI_API_KEYS=...
POCKET_TTS_ENABLED=true
POCKET_TTS_COMMAND=uvx
POCKET_TTS_COMMAND_ARGS=pocket-tts,generate
```

A sane order of work.

This roadmap avoids the tempting trap of adding shiny features before the launch foundation is ready.

0

Keep refining locally

Mac backend remains the fastest lab. Keep improving UI, voice timing, reminder feel, and memory quality.

1

Supabase production schema

Finalize tables, policies/server-only access, backup/export/delete strategy, and migration scripts.

2

Public backend alpha

Deploy a quick HTTPS backend. Rebuild IPA with remote API URL. Test STT, chat, voice, memory, reminders.

3

Always-on beta backend

Move to VM/VPS/Oracle. Add PM2/systemd, Caddy HTTPS, Pocket TTS cache, logs, uptime monitor.

4

Accounts and privacy

Add Apple Sign In or anonymous-to-account migration, privacy policy, delete/export controls.

5

TestFlight

Invite small group. Measure latency, cost, memory correctness, reminder reliability, emotional UX.

6

Monetization

Add BYOK and/or subscription caps. Do not open public unlimited usage before cost controls exist.

7

App Store

Submit with production backend, support, privacy, analytics, crash logs, and real operational monitoring.

The recommended future.

For now, AURA stays on the Mac. That is the correct move while the product is still being shaped. The serious future is a public HTTPS backend plus persistent memory, then TestFlight.

Do now

Keep Mac backend for fast testing. Keep improving AURA's feel, memory, reminders, and voice.

Do next

Create Supabase production schema and deploy a temporary HTTPS backend to prove the remote flow.

Do for beta

Use Oracle Always Free VM or a low-cost VPS for always-on backend and Pocket TTS. Then build TestFlight.

Do for launch

Add accounts, data controls, usage caps, billing/BYOK, support, monitoring, and App Store metadata.

Core principle: AURA is not just “chat with voice.” Its defensible magic is emotionally intelligent long-term memory. Protect reliability and trust before scale.

References: Render Free, Railway Pricing, Vercel Function Limits, Koyeb Pricing FAQ, Oracle Always Free, Supabase Pricing.

Generated for the current AURA prototype on May 16, 2026.